计算机图形学 · 课堂笔记

计算机图形学

2025年3月16日 11:51

第一章 诸论

计算机图形学基本概念：ISO：计算机图形学是研究通过计算机将数据转换成图形，并在专门显示设备上显示的原理、方法和技术的科学。

IEEE：计算机图形学是借助计算机产生图形影像的一门艺术和科学。

广泛采用：计算机图形学是研究利用计算机表示、生成、处理和显示图形的原理、算法、方法和技术的一门学科

计算机图形学的图形要素：几何要素、非几何要素

表示图形的方法：点阵式和矢量式

图形学的应用：用户接口（UI）、计算机辅助设计与制造（CAD/CAM）、娱乐(Entertainment)、计算机辅助绘图（Presentation Graphics）

名称	相应的典型设备	基本功能
定位	叉丝、拇指轮	输入一个点的坐标
笔画	图形输入板	输入一系列点的坐标
数值	数字键盘	输入一个整数或实数
选择	功能键、叉丝等	由一个整数得到某种选择
拾取	光笔或叉丝触摸屏幕上已显示的图形	通过一种拾取状态来判别一个显示着的图形
字符串	字符键盘	输入一个字符串

图 1

计算机辅助教学(Education and training)、科学计算的可视化(Visualization)、计算机艺术(Computer Art)

第二章 计算机图形系统

计算机图形系统：计算机硬件+图形输入输出设备+计算机系统软件+图形软件

功能：图形输入、图形计算、图形交互、图形输出、图形存储

图形输入设备的逻辑分类：

刷新式CRT：采用电子束周期性地回到同一点的方式来保持图像的CRT

原理：高速的电子束由电子枪发出，经过聚焦系统、加速系统和偏转系统就会到达荧光屏的特定位置。由于荧光物质在高速电子的轰击下会发生电子跃迁，即电子吸收到能量从低能态变为高能态，由于高能态很

不稳定，在很短的

	随机扫描显示器	光栅扫描显示器
电子束扫描方式	电子束只扫向图形所在的屏幕位置。一次一条线。 随机系统的刷新速率依赖于显示的线数；30~60HZ；100,000条 。	电子束自顶向下全屏扫描 逐行扫描/隔行扫描 刷新率、闪烁
图形定义	图形定义为一组画线命令，存储在被称为刷新显示文件的内存区域。	图形定义为一组屏幕点信息，存储在被称作帧缓冲区的内存区域。 Pixel (Picture element)像素点 帧缓冲区容量=显示分辨率x色深
适用范围	适于画线应用	适于逼真显示包含细微阴影和色彩模式的场景

图 2

时间内荧光物质的电子会从高能态重新回到低能态，这时将发出荧光，屏幕上产生亮点。要保持显示一幅稳定的画面，必须不断地发射电子束。

光点物理直径：光点亮度下降到最大亮度60%处的直径

光栅扫描显示器和随机扫描显示器是最常见的两种刷新式CRT

光栅扫描系统由帧缓冲存储器（保存一组对应屏幕所有点的强度值像素点）、视频控制器（基本刷新操作）、显示处理器构成

视频显示设备：刷新式CRT、光栅扫描和随机扫描、光栅扫描系统、彩色CRT、平板显示器

用户接口设计：用户接口确定用户与计算机如何进行信息交换。包括用户通过什么途径与图形系统进行联系，通过什么手段来操作系统的功能实现等。

手段：显示屏幕的有效利用（信息显示的布局合理性、充分而又正确地使用图符）、反馈、一致性原则、减少记忆量、回退和出错处理、联机帮助、视觉效果设计、适应不同用户

直线的斜率	直线上点的坐标 递推公式	逼近直线的 象素点的坐标
$\|k\|\leq 1$	$\begin{cases} x_{i+1}=x_i+1 \\ y_{i+1}=y_i+k \end{cases}$	$(x_{i+1},\text{round}(y_{i+1}))$
$\|k\|>1$	$\begin{cases} y_{i+1}=y_i+1 \\ x_{i+1}=x_i+1/k \end{cases}$	$(\text{round}(x_{i+1}), y_{i+1})$

图 3

$0\leqslant k\leqslant 1$时中点画线算法的算法步骤为：

1. 输入直线的两端点$P_0(x_0,y_0)$和$p_1(x_1,y_1)$。
2. 计算初始值$a=y_0-y_1$、$b=x_1-x_0$、$d=a+0.5b$、$x=x_0$、$y=y_0$；
3. 绘制点(x, y)。判断d的符号；
   若d<0，则(x, y)更新为(x+1, y+1)，d更新为d+a+b；否则(x, y)更新为(x+1, y)，d更新为d+a。
4. 当直线没有画完时，重复步骤3。否则结束。

图 4

改进：用2d代替d

程序

1. 输入直线的两端点$P_0(x_0,y_0)$和$p_1(x_1,y_1)$。
2. 计算初始值$a=y_0-y_1$、$b=x_1-x_0$、$d=2*a+b$、$x=x_0$、$y=y_0$、$d_1=2*a$、$d_2=2*(a+b)$。
3. 绘制点(x, y)。判断d的符号。
   若d<0，则(x, y)更新为(x+1, y+1)，d更新为$d+d_2$；
   否则(x, y)更新为(x+1, y)，d更新为$d+d_1$。
4. 当直线没有画完时，重复步骤3。否则结束。

图 5

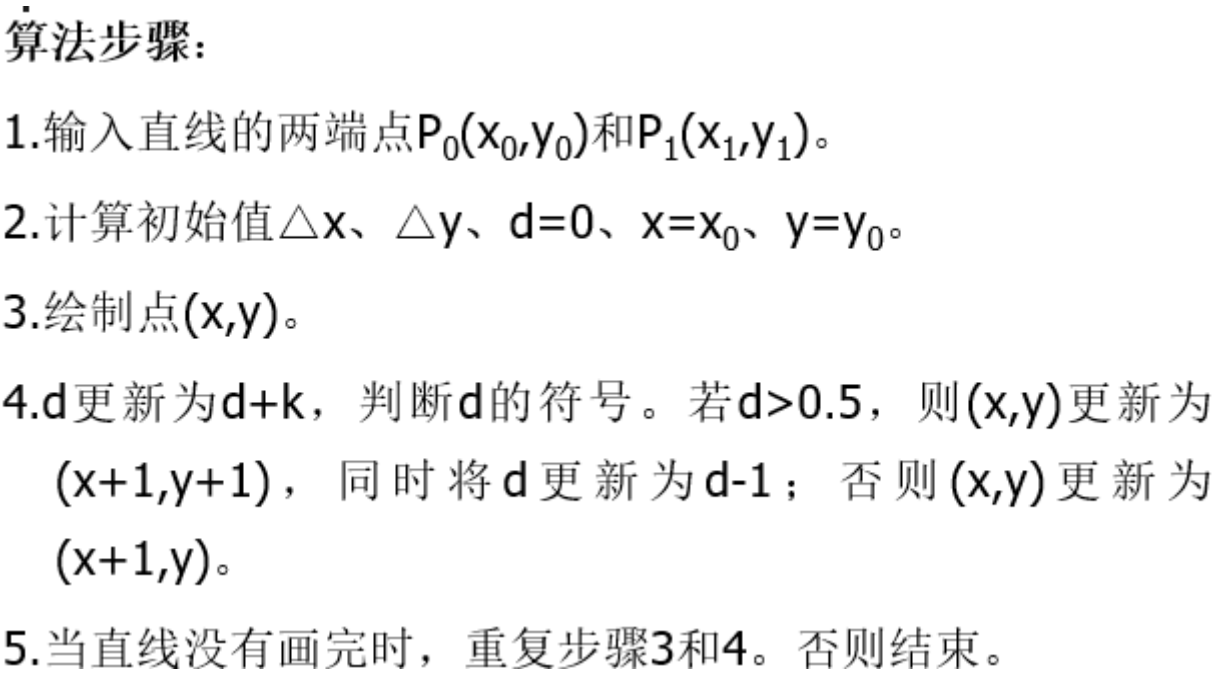


图 6

基本交互绘图技术：回显、约束、网络、引力域、橡皮筋技术、草拟技术、拖动、旋转、形变

第四章 基本图形生成算法

图形的扫描转换：在光栅显示器等设备上确定一个最佳逼近于图形的象素集的过程，是完成图形的参数表示形式到点阵表示形式的转换

直线的扫描转换：方法：DDA画线法、中点画线法、Bresenham画线算法

DDA画线法：通过同时对x和y各增加一个小增量，计算下一步的x、y值。是个增量算法

中点画线法：借助于一个决策变量 d的正负符号 ,来确定下一个该点亮的象素点。

Bresenham画线算法：

改进1：令e=d-0.5 改进2：用2eΔx替换e，e初=-Δx

算法步骤为：

1.输入直线的两端点$P_0(x_0,y_0)$和$P_1(x_1,y_1)$。

2.计算初始值△x、△y、**e=-0.5**、$x=x_0$、$y=y_0$。

3.绘制点(x,y)。

4.**e**更新为**e+k**，判断**e**的符号。若**e**>0，则(x,y)更新为(x+1,y+1)，同时将**e**更新为**e-1**；否则(x,y)更新为(x+1,y)。

5.当直线没有画完时，重复步骤3和4。否则结束。

图 7

算法步骤：

1.输入直线的两端点$P_0(x_0,y_0)$和$P_1(x_1,y_1)$。

程序

2.计算初始值△x、△y、e=-△x、$x=x_0$、$y=y_0$。

3.绘制点(x,y)。

4.e更新为e+2△y，判断e的符号。若e>0，则(x,y)更新为(x+1,y+1)，同时将e更新为e-2△x；否则(x,y)更新为(x+1,y)。

5.当直线没有画完时，重复步骤3和4。否则结束。

图 8

算法步骤：

1.输入圆的半径R。

2.计算初始值d=1.25-R、x=0、y=R。

3.绘制点(x,y)及其在八分圆中的另外七个对称点。

4.判断d的符号。若d≤0，则先将d更新为d+2x+3，再将(x,y)更新为(x+1,y)；否则先将d更新为d+2(x-y)+5，再将(x,y)更新为(x+1,y-1)。

5.当x<y时，重复步骤3和4。否则结束。

图 9

1.输入圆的半径R。

2.计算初始值d=1-R、x=0、y=R。

3.绘制点(x,y)及其在八分圆中的另外七个对称点。

4.判断d的符号。若d≤0，则先将(x,y)更新为(x+1,y)，再将d更新为d+2x+3；否则先将(x,y)更新为(x+1,y-1)，再将d更新为d+2(x-y)+5。

5.当x<y时，重复步骤3和4。否则结束。程序 实例

图 10

圆的扫描转换：中点画圆法、Bresenham画圆法

中点画圆法： 改进：将1.25-R约等于1-R（因为R为整数）

Bresenham画圆法：Pk-1<0，Pk=Pk-1+4Xk+6；Pk-1>=0，Pk=Pk-1+4（Xk-Yk）+10

1) 输入圆半径r和圆心(x_c, y_c)，
获得第一个点：$(x_0, y_0) = (0, R)$
2) 计算$P_0 = 3 - 2R$;
3) 根据公式计算P_{k+1}，确定下一点；
如果 $p_k < 0$, 选择 (x_{k+1}, y_k)
$p_k \geq 0$, 选择$(x_{k+1}, y_k - 1)$
4) 确定对称点；
5) 重复步骤3)，直至$x \geq y$

图 11

算法步骤：

1.输入椭圆的长半轴a和短半轴b。

2.计算初始值$d=b^2+a^2(-b+0.25)$、x=0、y=b。

3.绘制点(x,y)及其在四分象限上的另外三个对称点。

图 12

7.绘制点(x,y)及其在四分象限上的另外三个对称点。

8.判断d的符号。若$d \leq 0$，则先将d更新为$b^2(2x_i+2)+a^2(-2y_i+3)$，再将(x,y)更新为(x+1,y-1)；否则先将d更新为$d+a^2(-2y_i+3)$，再将(x,y)更新为(x,y-1)。

9.当y>0时，重复步骤7和8。否则结束。 程序 实例

图 13

$$d = b^2(x+0.5)^2 + a^2(y-1)^2 - a^2b^2$$

4.判断d的符号。若d≤0，则先将d更新为d+b²(2x+3)，再将(x,y)更新为(x+1,y)；否则先将d更新为d+b²(2x+3)+a²(-2y+2)，再将(x,y)更新为(x+1,y-1)。

5.当b²(x+1)<a²(y-0.5)时，重复步骤3和4。否则转到步骤6。

6.用上半部分计算的最后点(x,y)来计算下半部分中d的初值：

图 14

椭圆的扫描转换

扫描线多边形填充算法：扫描线自底向上扫描，计算扫描线与多边形边界的交点确定填充区间，再用要求的颜色显示这些区间的像素，即完成填充工作。一条扫描线的填充过程分为求交、排序、配对和填色四个步骤。

算法描述

算法步骤

1)输入多边形顶点数及顶点坐标；

2)建立有序边表；

3)根据当前扫描值建立活化边表；

4)求出扫描线与多边形边界交点，交点配对、填充；

5)更新活化边表并重新排序；

6)进入下一条扫描线，重复步骤3，直至扫描线值为窗口高度。

图 15

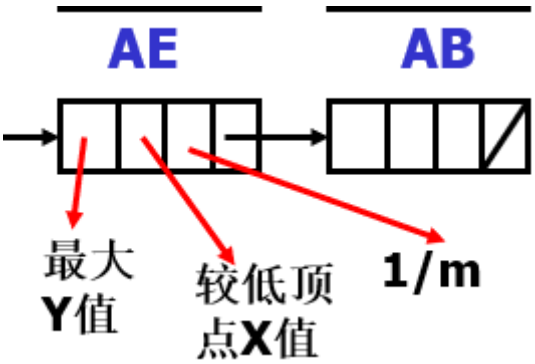


图 16

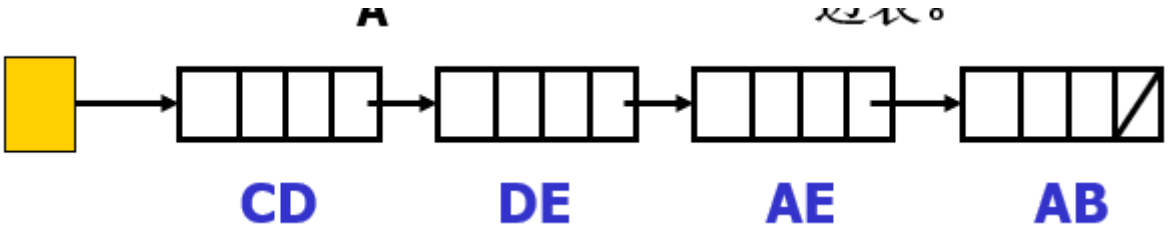


图 17

有序边表： 活化边表：

边缘填充算法：帧缓冲器中对多边形的每条边进行直线扫描转换，亦即对多边形边界所经过的象素打上标志。

然后再采用和扫描线算法类似的方法将位于多边形内的各个区段着上所需颜色。使用一个布尔量inside来指示当前点是否在多边形内的状态。

种子填充算法：分为边界填充算法和泛填充算法

边界填充算法：填充区域边界以一种颜色指定；从区域的一个内部点开始，由内至外绘制直到边界。适用于单色边界。

填充方式：4-连通、8-连通

4-连通：种子象素入栈；当栈非空时重复执行如下三步操作：

(1)栈顶象素出栈；(2)将出栈象素置成填充色；(3)检查出栈象素的4-邻接点，若其中某个象素点不是边界色且未置成多边形色，则把该象素入栈。

8-连通：种子象素入栈；当栈非空时重复执行如下三步操作：

(1)栈顶象素出栈；(2)将出栈象素置成填充色；(3)检查出栈象素的8-邻接点，若其中某个象素点不是边界色且未置成多边形色，则把该象素入栈。

沿扫描线填充水平像素段的4-连通：

种子象素入栈；当栈非空时作如下三步操作：

(1)栈顶象素出栈；

(2)填充出栈象素所在扫描行的连续象素段，直到遇到边界象素为止，即每出栈一个象素，就对包含该象素的整个扫描线区间进行填充；

(3)在区间中检查与当前扫描线相邻的上下两条扫描线的有关象素是否全为边界象素或已填充的象素，若存在非边界、未填充边界的象素，则把每一区间的最右（或最左）象素取作种子象素入栈

泛填充算法：填充区域以一种颜色指定；从指定的内部点 (x,y) 开始，将填充色赋给颜色为给定内部色的邻接像素。适用于多色边界

8-连通：种子象素入栈；当栈非空时重复执行如下三步操作：

(1)栈顶象素出栈；(2)将出栈象素置成填充色；(3)检查出栈象素的8-邻接点，若其中某个象素点不是给定内部点的颜色且未置成新的填充色，则把该象素入栈

当以边界表示时，4-连通边界填充算法只能填充4-连通区域，8-连通边界填充算法也只能填充8-连通区域。当以内点表示时，8-连通泛填充算法可以填充8-连通区域也可以填充4-连通区域，当然4-连通泛填充算法还是只能填充

4-连通区域。

奇-偶规则：从任意位置p作一条射线，若与该射线相交的多边形边的数目为奇数，则p是多边形内部点，否则是外部点。

非零环绕数规则：首先使多边形的边变为矢量。将环绕数初始化为零。再从任意位置p作一条射线。当从p点沿射线方向移动时，对在每个方向上穿过射线的边计数，每当多边形的边从右到左穿过射线时，环绕数加1，从左到右

时，环绕数减1。处理完多边形的所有相关边之后，若环绕数为非零，则p为内部点，否则，p是外部点。

属性处理：

线型处理：可根据线的斜率来调整实心段和中间空白段的象素数目。

线宽处理：类型：(1)水平或垂直方向扩展(2) 矩形区域填充 (3) 画笔或画刷

水平或垂直的方法处理线宽的优缺点：

（1）实现简单、效率高。

（2）斜线与水平(或垂直)线不一样粗。当线宽为偶数个象素时，线的中心将偏移半个象素。

（3）利用线刷子生成线的始末端总是水平或垂直的，看起来不太自然。

解决方法：添加“线帽”，分为方帽，突方帽，圆帽

常见的画刷是方刷子，它通过把边长为指定线宽的正方形的中心沿直线做平行移动来获取具有宽度的线条

曲线的线型：根据曲线斜率设置像素掩码的实心段和空白段像素数目

曲线的线宽：水平扩展、垂直扩展和画刷

字符的属性：字体、字高、自宽、字倾斜角、对齐、字色.....

点阵字符：采用逐位映射的方式得到字符的字模编码 特点：易于定义显示、空间需求量大

矢量字符：将字符笔画分解为线段，以线段端点坐标为字符字模的编码 特点：空间需求量小、时间开销大

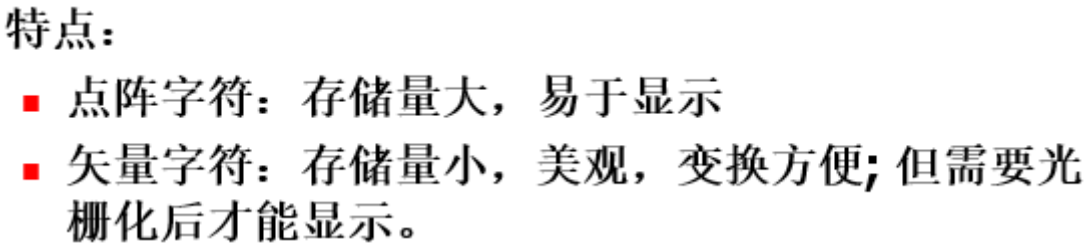


图 18

走样：用离散量表示连续量引起的失真

反走样：用于减少或消除这种效果的技术 方法：提高分辨率、简单区域取样、加权区域取样

提高分辨率：方法简单，代价非常大。侦缓存容量和扫描时间增加3倍。只能减轻不能消除锯齿问题。

可用软件方法提高分辨率——过取样：把屏幕看成比实际所具有的更细的网格来增加取样率

简单区域取样：每个象素是一个具有一定面积的小区域，将直线段看作具有一定宽度的狭长矩形。当直线段与象素有交时，求出两者相交区域的面积，然后根据相交区域面积的大小确定该象素的亮度值

近似面积的离散计算方法：(1)将屏幕象素分割成n个更小的子象素，

(2)计算中心落在直线段内的子象素的个数m，

(3)m/n为线段与象素相交区域面积的近似值。

缺点：（1）象素的亮度与相交区域的面积成正比，而与相交区域落在象素内的位置无关，这仍然会导致锯齿效应。

（2）直线条上沿理想直线方向的相邻两个象素有时会有较大的灰度差。

加权区域取样：假想一个连续的加权曲面（或过滤函数）覆盖象素。当直线条经过该象素时，该象素的灰度值是在二者重叠区域上对滤波器（过滤函数）进行积分的积分值。

$$P=\begin{pmatrix} x \\ y \end{pmatrix} \quad P'=\begin{pmatrix} x' \\ y' \end{pmatrix} \quad R=\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

$$P' = R \cdot P$$

图 19

$$P=\begin{pmatrix} x \\ y \end{pmatrix} \quad P'=\begin{pmatrix} x' \\ y' \end{pmatrix} \quad S=\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$$

$$P' = S \cdot P$$

图 20

特点：（1）接近理想直线的象素将被分配更多的灰度值；

（2）相邻两个象素的滤波器相交，有利于缩小直线条上相邻象素的灰度差。

第五章 二维变换和二维观察

基本变换：平移、旋转、变比

旋转：针对坐标原点，x’=x*cosθ- y*sinθ ，y’= x*sinθ+ y*cosθ

变比：

引入齐次坐标后的平移、旋转、变比

复合变换：连续平移、连续旋转、连续变比、通用基准点的变换、通用方向的变换

连续平移是可加的 连续旋转也是可加的，将θ换成θ1+θ2即可 连续变比是相乘的，将Sx和Sy变成Sx1*Sx2和Sy1*Sy2

通用基准点的变换步骤：平移使基准点移动到坐标原点(T)，针对原点做指定变换(M)，反向平移使基准点回到原始位置(T-1)

$$P' = P+T \text{ (平移)};$$
$$P' = R \cdot P \text{ (旋转)};$$
$$P' = S \cdot P \text{ (变比)};$$

图 21

平移变换

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$P' = T(t_x, t_y) * P$$

图 22

旋转变换

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$P' = R(\theta) * P$$

图 23

变比变换

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$P' = S(s_x, s_y) * P$$

图 24

（1）沿 x 轴方向错切

变换矩阵为 $\begin{bmatrix} 0 & SH_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

（2）沿 y 轴方向错切

变换矩阵为 $\begin{bmatrix} 1 & 0 & 0 \\ SH_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

图 25

通用方向变换：旋转对象使任意方向与坐标轴方向重合，针对坐标轴方向做指定变换，反向旋转使任意方向回到原方向

通用复合变换矩阵：

其他变换：反射（对称）、错切

对称：关于x轴对称： 关于y轴对称： 关于原点对称： 关于y=x对称：

错切：

坐标系间的变换：观察夹角，先平移到坐标原点再旋转

窗口vs视区： (Xw-W1) / (W2-W1) = (Xv-V1) / (V2-V1) (Yw-W3) / (W4-W3) = (Yv-V3) / (V4-V3)

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} rs_{xx} & rs_{xy} & trs_x \\ rs_{yx} & rs_{yy} & trs_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

图 26

算法描述

- 计算直线端点编码, c1 和 c2;
- 判断
  - c1 和 c2 均为0000，保留直线
  - c1 & c2 不为零，同在某一边界外，删除该直线
  - c1 和 c2不 均为0000 且c1 & c2 为零，需要进一步求解交点
- 以L,R,B,T 为序，将x=w1/w2或y=w3/w4带入直线方程，计算直线与窗口边界的交点，将交点和另一端点重复上述过程，直至线段保留或删除

图 27

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

图 28

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

图 29

$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

图 30

$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

图 31

裁剪：识别图形在指定区域内或区域外的过程，针对窗口边界裁剪、针对视区边界裁剪

点裁剪：判断点是否在窗口内

直线裁剪：实交点是直线段与窗口矩形边界的交点。虚交点则是直线段与窗口矩形边界延长线或直线段的延长线与窗口矩形边界的交点。

CS直线裁剪算法：直线由端点标识；测试直线端点和窗口边界的关系以确定是否需要计算交点

LB直线裁剪算法： 基于直线段参数方程分析的快速直线裁剪算法。

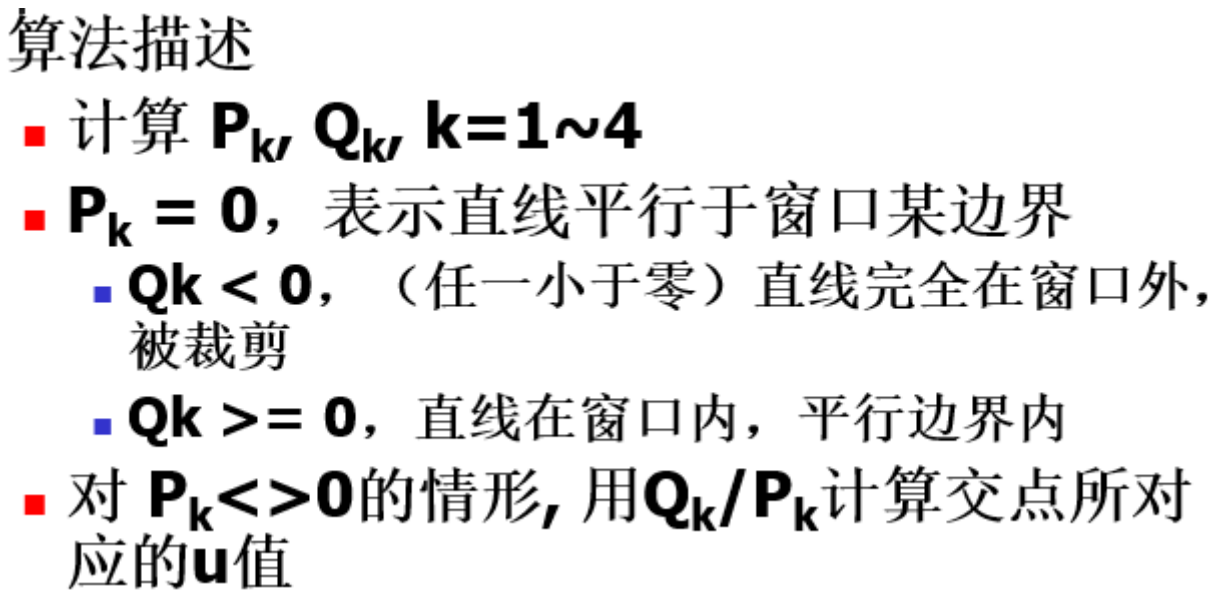


图 32

中点分割裁剪算法：通过二分逼近来确定直线段与窗口的交点

NLN直线裁剪算法：通过在裁剪窗口周围创立多个区域，并在求交运算之前进行更多的区域测试，从而避免对直线段进行多次剪裁。

多边形裁剪算法：Sutherland-Hodgman 算法、Weiler-Atherton 算法

Sutherland-Hodgman 算法：以多边形顶点为初始集合, 首先用窗口左边界剪裁多边形，产生新的顶点序列。新的顶点集依次传给右边界、下边界和上边界进行处理

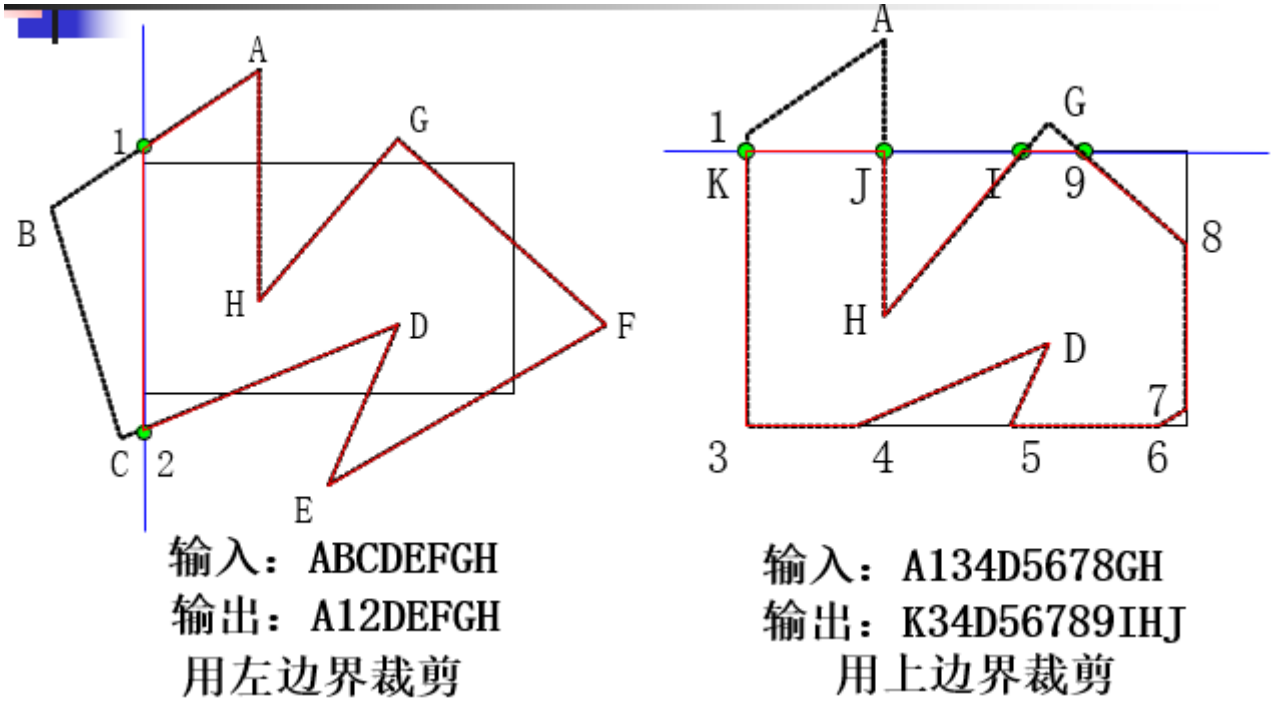


图 33

如果直线在窗口内，则

$$w_1 \le x_1 + dx * u \le w_2$$

$$w_3 \le y_1 + dy * u \le w_4$$

统一表示为：$P_k * u \le Q_k \quad k = 1, 2, 3, 4$

$$P_1 = -dx,\ Q_1 = x_1 - w_1 \quad P_2 = dx,\ Q_2 = w_2 - x_1$$

$$P_3 = -dy,\ Q_3 = y_1 - w_3 \quad P_4 = dy,\ Q_4 = w_4 - y_1$$

图 34

- 对每条线计算参数u_1&u_2

$$u_1=Max(\{Q_k/P_k \mid P_k < 0\} \cup \{0\})$$

$$u_2=Min(\{Q_k/P_k \mid P_k > 0\} \cup \{1\})$$

- 如果u1 > u2, 则直线在窗口外，否则计算交点坐标

$$x(u) = x1+dx*u$$

$$y(u) = y1+dy*u$$

图 35

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

图 36

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

图 37

Weiler-Atherton 算法：通过修改多边形顶点处理顺序(考虑顶点处理方向)，从而正确显示凹多边形。

第六章 三维变换和三位观察

3D变换：3D平移、3D变比、3D旋转、3D反射与错切、3D复合变换

3D平移：

3D变比：

3D旋转： x轴： y轴： z轴：

常规3D旋转：

（1）任意轴平行于坐标轴之一：平移使任意轴与平行坐标轴重合，完成指定旋转，反向平移使回到原位置 变换矩阵：M=T-1 R(θ)T

（2）任意轴不平行于任何坐标轴：平移使任意轴过原点 ，旋转使任意轴与坐标轴之一重合，完成指定旋转，反向旋转，反向平移 变换矩阵：M=T-1Rx(-a)Ry(-b)Rz(θ)Ry(b)Rx(a)T

(1)使任意方向轴的起点与坐标原点重合，此时进行平移变换。

(2)使方向轴与某一坐标轴重合，此时需进行旋转变换，且旋转变换可能不止一次。

(3)针对该坐标轴完成变换。

(4)用逆旋转变换使方向轴回到其原始方向。

(5)用逆平移变换使方向轴回到其原始位置。

图 38

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

图 39

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

图 40

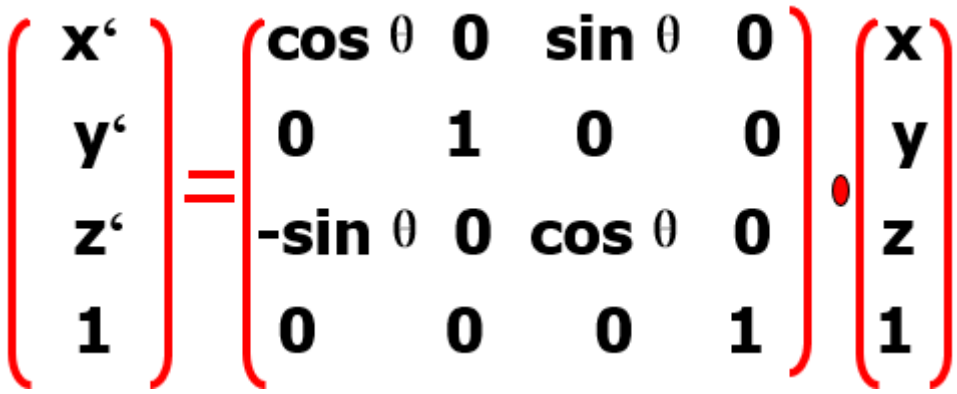


图 41

3D反射：xOy平面： 其余同理

3D错切：沿z轴：

三维复合变换：

（1）相对任一参考点的三维变换：将参考点F移至坐标原点，针对原点进行二维几何变换，进行反平移

（2）绕任意轴的三维旋转变换：

三维投影变换：把三维物体变为二维图形表示的过程称为投影变换。

分类：平行投影：坐标位置沿平行线变换到观察平面上。

透视投影：物体位置沿收敛于某点的直线变换到观察平面上，该点称为投影参考点。

平行投影：只需给出投影方向即可，分为正投影和斜投影

正投影分为三视图（主：XOZ，侧（绕z轴正方向旋转90度）：YOZ，俯（绕x轴负方向旋转90度）：XOY）和正轴测（投影面和坐标轴呈一定关系）

$$
\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \boldsymbol{T}_h \boldsymbol{R}_x(\beta) \boldsymbol{T}_y(-\alpha) \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}
$$

$$
= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\beta & -\sin\beta & 0 \\ 0 & \sin\beta & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\alpha & 0 & -\sin\alpha & 0 \\ 0 & 1 & 0 & 0 \\ \sin\alpha & 0 & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}
$$

$$
= \begin{bmatrix} \cos\alpha & 0 & -\sin\alpha & 0 \\ -\sin\alpha\sin\beta & \cos\beta & -\cos\alpha\sin\beta & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}
$$

$$
= \begin{bmatrix} x\cos\alpha - z\sin\alpha \\ y\cos\beta - \sin\beta(x\sin\alpha + z\cos\alpha) \\ 0 \\ 1 \end{bmatrix}
$$

图 42

$$
RFz = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}
$$

图 43

$$SHz = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ a & b & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

图 44

正轴测：正等测（投影平面与三个坐标轴的交点到坐标原点的距离都相等）、正二测（交点到原点距离有两个相等）、正三测（都相等）

形成过程：将空间一立体绕y轴旋转θy角，然后再绕x轴旋转θx，最后向z=0平面做正投影

斜投影： L1=1/tana

斜等测：a=45度 斜二测：a=arctan(2)

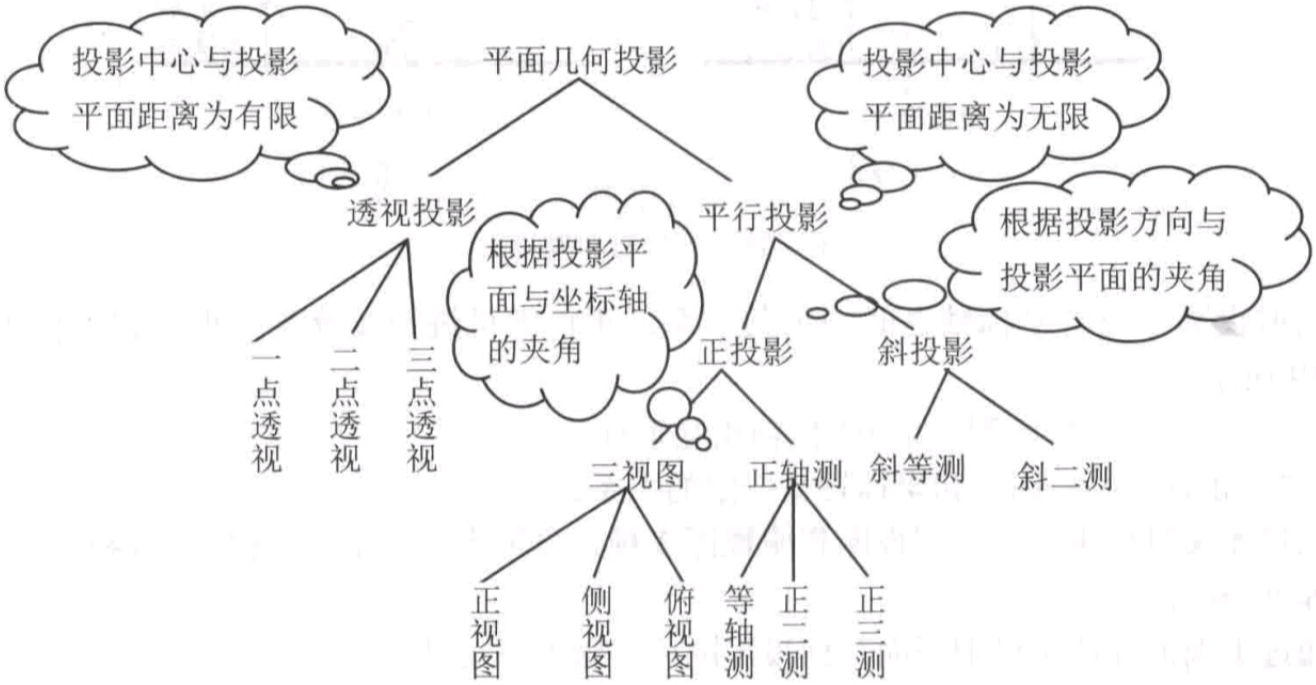


图 45

透视投影：空间不平行于画面(投影面）的一切平行线的透视投影，分为一点透视、两点透视、三点透视

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -\frac{1}{h} & 1 \end{bmatrix}$$

图 46

灭点：不平行于投影平面的平行线，经过透视投影之后收敛于一点

主灭点:平行于坐标轴的平行线产生的灭点。

一点透视： 二点透视： 三点透视：

一点透视变换矩阵：

视点在(0,0,h),投影z轴 (0,h,0),投影y轴 (h,0,0),投影x轴

一点透视为了避免投影变换后一些平面或直线汇聚成直线或点，要考虑将参照物适当平移（平移变换）

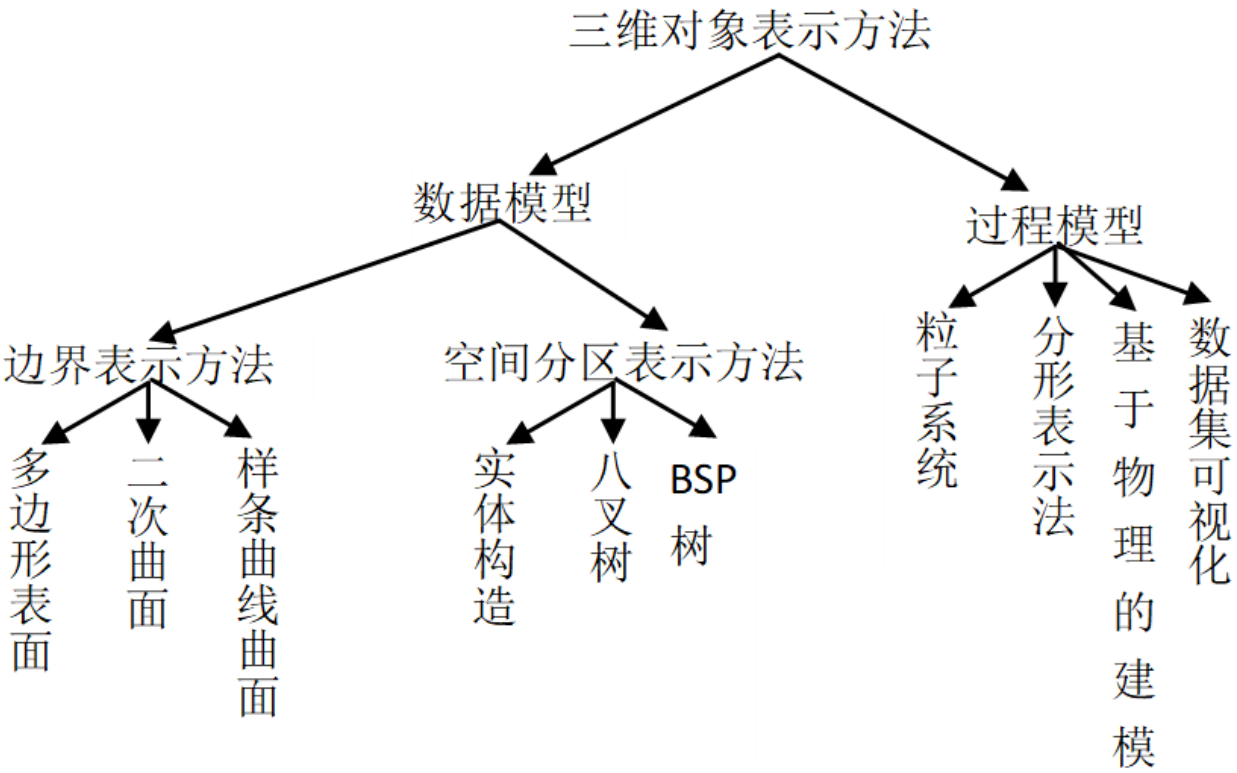


图 47

$$M = \begin{bmatrix} 1 & 0 & L_1\cos\varphi & 0 \\ 0 & 1 & L_1\sin\varphi & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

图 48

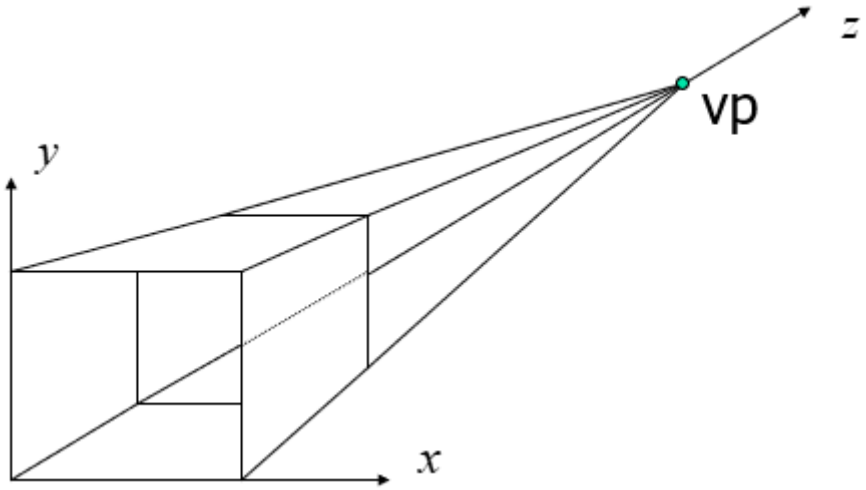


图 49

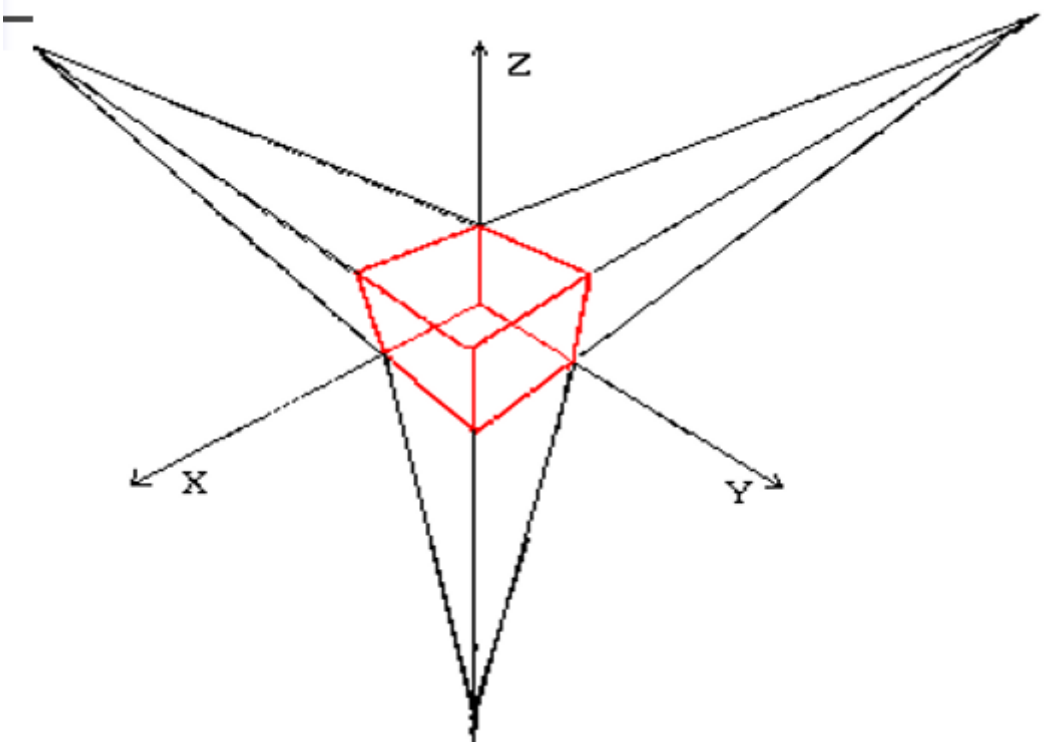


图 50

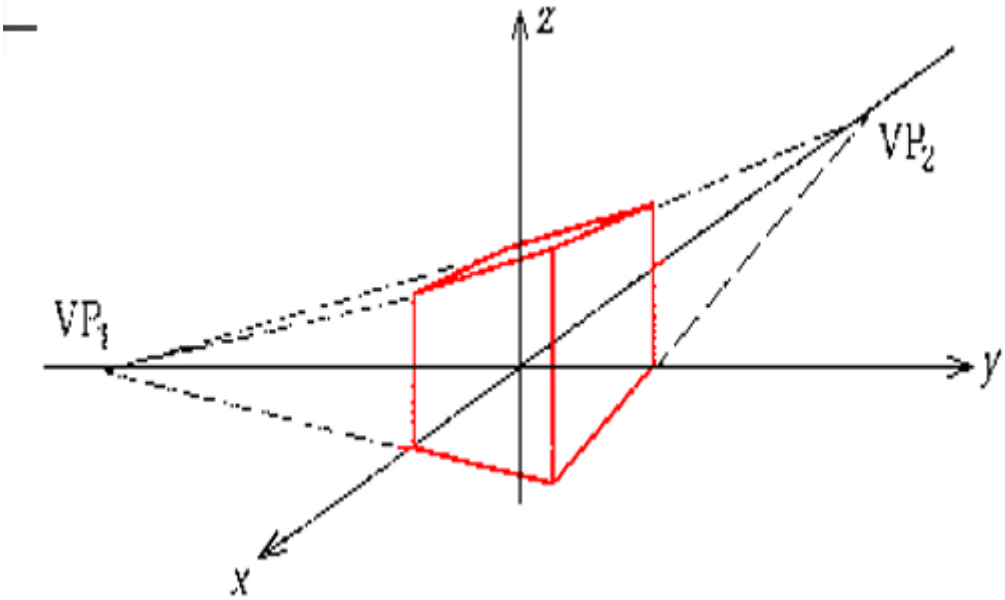


图 51

二点透视：当变换矩阵中第四列的p，q，r有两个不为0时的透视变换，两个灭点

三点透视：p，q，r都不为0，三个灭点

第七章 三维对象的表示

第八章 消隐

消隐：在给定视点和视线方向后，决定场景中哪些物体的表面是可见的，哪些是被遮挡不可见的，这一过程称为物体的消隐。

分类：按消隐的对象：线消隐（物体上不可见的线）、面消隐（物体上不可见的面）

按消隐的空间：图像空间（屏幕坐标系）的消隐：确定距离视点最近的物体，以该物体表面的颜色来显示像素 Z缓冲区

景物空间（观察坐标系）的消隐：将空间中每一个物体与其他物体进行比较，确定遮挡关系，显示该物体表面的可见部分 BSP

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -\frac{1}{h} & 0 & 1 \end{bmatrix}$$

图 52

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -\frac{1}{h} & 0 & 0 & 1 \end{bmatrix}$$

图 53

当$z \to \infty$时，$x' \to 0, y' \to 0, z' \to -h$

$\therefore (0,0,-h)$为该透视的一个灭点。

同样，视点在$(h,0,0)$的透视变换，灭点在$(-h,0,0)$

图 54

原则：排序、连贯性、

Z缓冲区算法（深度缓冲区算法）：对给定的x，y，寻找最小的z（x，y），最终深度缓冲器中保存的是可见面的深度值， 刷新缓冲器中保存的是这些表面的对应属性值。

解：如下图所示，假设 xOy 面为投影面（屏幕坐标系），由于通常选择 Z 轴的负方向为观察方向，因此算法沿着观察系统的 Z 轴来计算各景物距离观察平面的深度（Z 坐标值），所以该算法称作 Z 缓冲区（Z-buffer）算法。下面结合图 8-3 和图 8-4 说明一下 Z 缓冲区算法的原理。：

（1）初始化。首先将帧缓冲器中全部存放背景色，深度缓冲器中所有的单元初始值设置为最小值。

（2）在把物体表面相应的多边形扫描转换成帧缓冲器中的信息之后，对于多边形上的每一个采样点（x,y）进行以下几步处理：

①计算采采样点(x,y)的深度值 z(x,y)。

②如果 z(x,y)大于 z 缓冲器中在(x,y)处的值，说明多边形(x,y)处的点比帧缓冲器中(x,y)处现在具有颜色的点更靠近观察者，因此要把 z(x,y)存入 z 缓冲器中的（x,y）处，再把多边形在 z(x,y)c 处的颜色值存入帧缓冲器的（x,y）位置中。

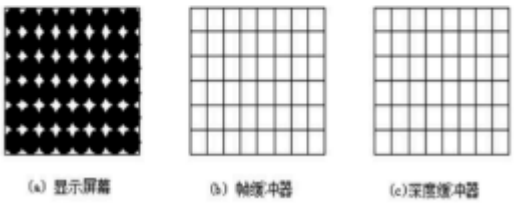


图 8-3 显示屏幕、帧缓冲器和深度缓冲器对应关系图

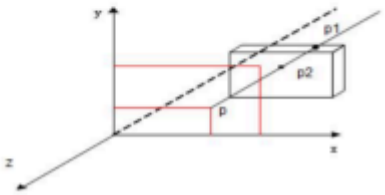


图 8-4 Z 缓冲区算法原理

图 55

曲线和曲面的表示

(1) 插值和逼近样条

(2) Hermite样条曲线

(3) Bezier曲线曲面

(4) B样条曲线曲面

(5) 有理样条曲线曲面

(6) 曲线曲面的转换和计算

图 56

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

图 57

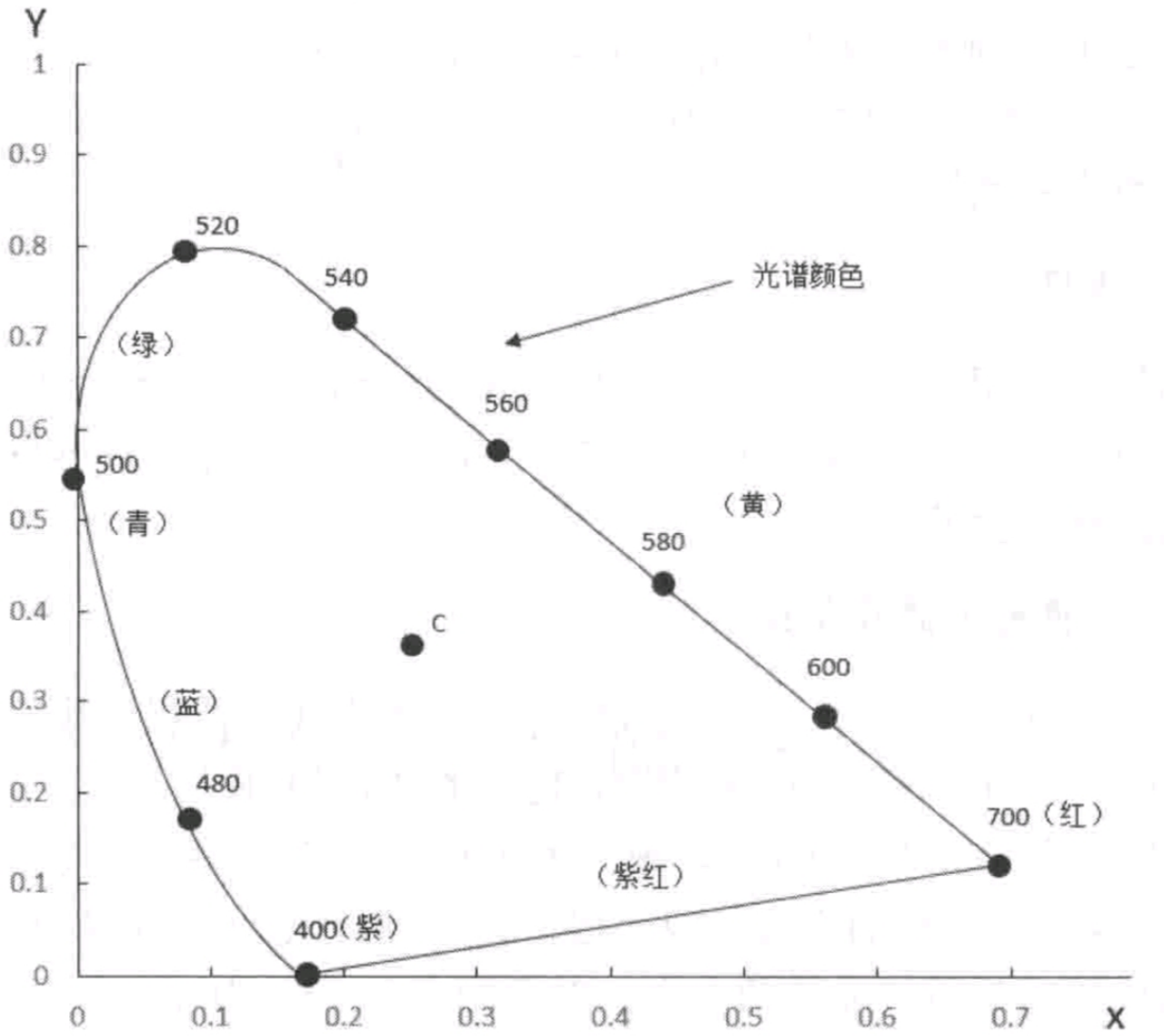


图 58

深度排序算法：将表面按深度递减方向排序；由深度最大的表面开始，逐个对表面进行扫描转换

第九章 颜色模型

RGB颜色模型：用红、绿和蓝三种颜色作为基色，加色模型

CMY颜色模型：一种减色模型，通过反射光看见颜色

HSV颜色模型：色彩角、饱和度、亮度

CIE色度图：

第十章 真实感图形绘制

简单光照模型：考虑反射光的作用，而不考虑透射光，假设物体不透明，那么物体表面呈现的颜色仅由其反射光决定

$$f(d)=\min\left\{1,\frac{1}{c_0+c_1d+c_2d^2}\right\}$$

图 59

从上面的讨论可以看出，Gouraud 明暗处理方法的优点是算法简单，计算量小，能有效地现实漫反射曲面。但该方法也存在如下缺点：

（1） 线性光强插值会造成表面出现过亮或过暗的条纹，即马赫带（Mach band）效应没有完成消除。

（2） 该方法只考虑了漫反射，对镜面反射效果不理想。也就是说，高光区域形状不规整，且只能在顶点范围形成，不能在多边形域内形成。

（3） 当对曲面采用不同的多边形进行分割时会产生那个不同的颜色效果。

图 60

整体光照模型：把处理物体之间光照的相互作用的模型

反射光 = 环境反射光+漫反射光+镜面反射光

光强衰减：距离光源近的景物比距离光源远的景物看起来更亮一些，光强的衰减和光源距离景物的距离d有关系

针对恒定光强的明暗处理：Gouraud明暗处理方法：对多边形顶点的颜色进行差值以便产生中间各点的颜色

Phong明暗处理方法：对多边形顶点的法矢量进行插值以便产生中间各点的法矢量

Phong明暗处理方法的基本思想是通过对多边形顶点的法矢量进行插值，获得其内部各点的法矢量，具体步骤如下：

从上面的讨论可以看出，采用 Phong 明暗处理方法绘制的图形比 Gouraud 明暗处理方法绘制的图形更真实，能够产生正确的高光区域。但由于 Phong 明暗处理方法不仅要计算各顶点的法矢量来插值计算多边形上各点的法矢量，还要用光照模型计算各点的光强值，因此其计算量远大于 Gouraud 明暗处理方法。

图 61

计算每个多边形的单位法矢量；

计算多边形顶点的单位法矢量；

对多边形顶点法矢量进行双线性插值，获得内部各点的法矢量；

利用光照模型计算多边形内部各顶点的颜色。

物体的表面细节称为纹理，纹理包括颜色纹理和几何纹理两类。

颜色纹理主要是指光滑表面上附加花纹和图案

几何纹理主要是指景物表面在微观上呈现出的起伏不平

图示与公式

7. 简述 RGB、CMY 和 HSV 颜色模型的联系和区别。（10 分）

解：

RGB 颜色模型用红、绿和蓝三种颜色作为基色，它是彩色显示器上显示彩色的基础，是一种加色模型。它是由 R，G 和 B 坐标轴定义的单位立方体。坐标原点代表黑色，其对角坐标点（1,1,1）代表白色。三个坐标轴上的顶点代表 3 个基色，而剩下的 3 个顶点则代表每一个基色的补色。RGB 颜色模型是一个加色模型，即多种基色的强度加在一起生成另一种颜色。立方体边界中的每一个颜色点都可以表示成三基色的加权向量和。用单位向量 R，G 和 B 表示为： $C(\lambda)=r\cdot R+g\cdot G+b\cdot B$ 。

CMY 颜色模型是由青色、品红和黄色作为三基色的单位立方体，也就是 RGB 模型的补色模型。在 CMY 模型中的坐标表示减去了 RGB 模型中的量是多少。CMY 颜色模型和 RGB 颜色模型存在如下式的转换关系，这里的单位列向量表示白色。

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

HSV 颜色模型中的 H，S 和 V 分别表示色彩角（H）、饱和度和亮度，它是从 RGB 立方体演变而来的。HSV 颜色模型对于多数用户是一个更加简单直观的模型。从指定一种纯色开始，即指定色彩角 H 并让 V=S=1.0，可以通过将白色或黑色加入到纯色中来描述所要的颜色。添加黑色则减小 V 而 S 保持不变。

图 62

计算机图形学 · 例题

计算机图形学例题

第二章

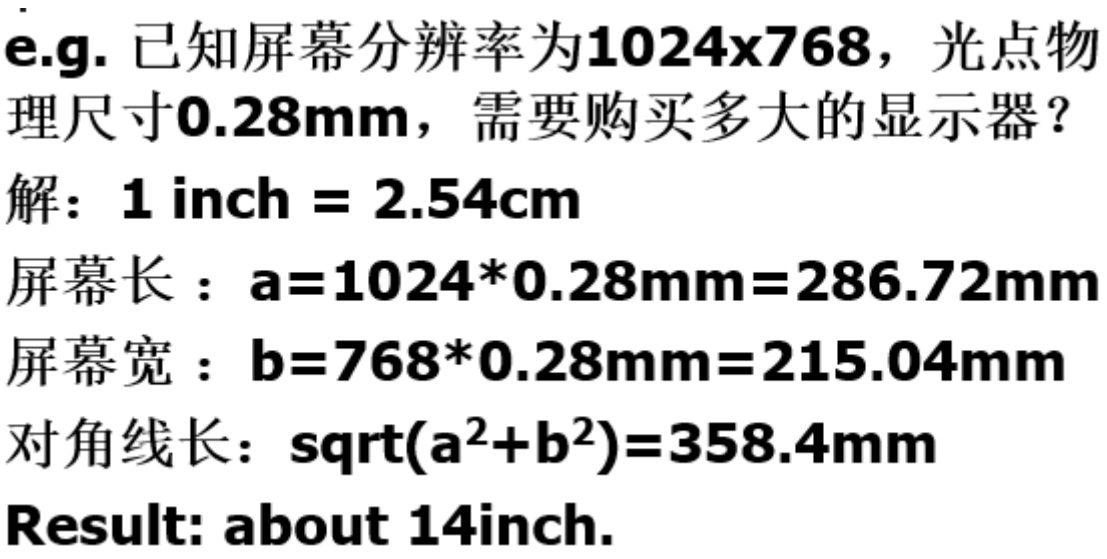


图 1

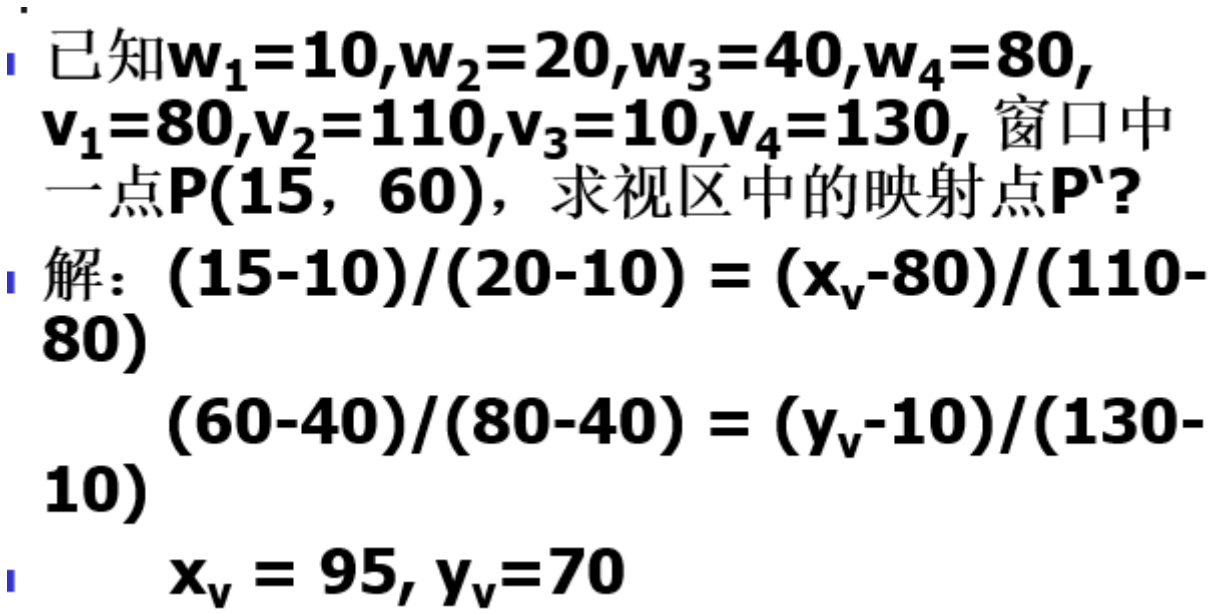


图 2

- 已知线段的两个端点$P_1(3, 4)$，$P_2(8, 2)$

  窗口边界$x=1, x=4, y=1, y=3$

- 用LB算法对线段进行裁剪

图 3

- 线段的参数方程 $x = 3 + 5u$

  $y = 4 - 2u$

- $P_1 = -5,\ Q1 = 2,\ R1 = -2/5$

  $P_2 = 5,\ Q2 = 1,\ R2 = 1/5$

  $P_3 = 2,\ Q3 = 3,\ R3 = 3/2$

  $P_4 = -2,\ Q4 = -1,\ R4 = 1/2$

- $u_1 = \max(0, -2/5, 1/2) = 1/2$

  $u_2 = \min(1, 1/5, 3/2) = 1/5$

- $u_1 > u_2$ 所以线段全部被裁剪

图 4

第五章最后几页ppt